

NEXUS AI : Agentic Software-Defined Vehicle Engineering and Validation Platform

From Requirement to Simulated Vehicle: Powered by Collaborative AI Agents

AD23731 Foundations of Agentic AI – Project Report

Submitted by

KOWSHIKA I R (2116231501080)
KRISHNA KUMAR N K (2116231501081)
LAKSHIYA SRI K M (2116231501082)

in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Under the guidance of

Dr. M. SOUNDARYA
ASSISTANT PROFESSOR



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AND MACHINE LEARNING**

**RAJALAKSHMI ENGINEERING COLLEGE
(AUTONOMOUS),
CHENNAI-602 105
OCTOBER 2026**

ACADEMIC YEAR 2026-2027



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report **“Nexus AI: Agentic Software Defined Vehicle Engineering and Validation Platform”** is the bonafide work of KOWSHIKA I R (231501080), KRISHNA KUMAR N K (231501081), LAKSHIYA SRI K M (231501082) VII Semester, B.Tech Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

I. INTRODUCTION.....	1
A. Background and Motivation	1
B. Need for Agentic AI	1
C. Project Objectives	2
II. PROBLEM STATEMENT AND BUSINESS CASE.....	2
A. Problem Definition	2
B. Business and Real-World Context	3
III. USER AND STAKEHOLDER ANALYSIS	3
A. Target Users.....	3
B. Stakeholder Identification.....	3
C. User Requirements.....	3
IV. PROPOSED AGENTIC AI SOLUTION	4
A. Overall Solution.....	4
B. Key Features	4
C. Agent-Based Approach.....	5
V. DATASET AND KNOWLEDGE SOURCES	5
A. Dataset or Knowledge Source Description	5
VI. TOOLS AND TECHNOLOGIES USED	6
VII. AGENT ARCHITECTURE	6
A. Agent Components	7
B. Tools	7
C. Memory.....	7
D. Reasoning Flow	8
E. Agent Interactions.....	8
VIII. AGENT WORKFLOW AND ORCHESTRATION	8
A. Workflow Design	8
B. Task Decomposition	9
C. Agent Coordination.....	9
D. Orchestration Implementation	9
IX. AGENTIC RAG IMPLEMENTATION	9
A. Retrieval Architecture.....	9
B. Knowledge Retrieval	10
C. Generation Process	10
D. Retrieval-Quality Evaluation	10
X. SYSTEM IMPLEMENTATION.....	10
A. Functional Prototype Screenshots.....	10
B. Source-Code Repository	14
XI. EXPERIMENTAL EVALUATION AND RESULTS.....	14
A. Evaluation Metrics.....	14
B. Testing Methodology	14
C. Performance Results	15
D. Success-Metric Achievement	16
XII. COMPARISON, INNOVATION AND SCALABILITY	16
A. Comparison with Baseline	16
B. Advantages of the Proposed System	17
C. Innovation	17
D. Scalability	17
E. Real-World Applicability.....	17
XIII. CONCLUSION AND FUTURE WORK.....	17
REFERENCES	18
INDEX OF KEY TERMS	19

LIST OF FIGURES

Fig. 1	Layered architecture of NEXUS AI.	7
Fig. 2	Agent workflow with reflection loop and HITL gate.	8
Fig. 3	NEXUS AI dashboard showing the multi-agent SDV engineering suite, the HITL audit gate, and live simulation telemetry.	11
Fig. 4	Eight-stage validation pipeline with Stage 1 hybrid RAG retrieval and ground-truth citations (agent ORION).	11
Fig. 5	Live vehicle cockpit with road visualiser, digital cockpit cluster, VSS telemetry trends, and fault-injection matrix.	12
Fig. 6	SDV project lifecycle and validation status page with the interactive project state controller.	12
Fig. 7	Safety validation and engineering audit report page with summary metrics and report sections.	13
Fig. 8	RAG SDV Copilot: standards question-answering assistant with hybrid Qdrant and BM25 grounding.	13
Fig. 9	Injected thermal fault profile (linear interpolation between the stated stage values). ...	15
Fig. 10	Composite score of the three configurations.	16

LIST OF TABLES

TABLE I	STAKEHOLDERS AND THEIR INTERESTS	3
TABLE II	FUNCTIONAL AND NON-FUNCTIONAL USER REQUIREMENTS	4
TABLE III	VSS 4.0 SIGNAL MODEL USED IN THE EV BATTERY SHOWCASE	5
TABLE IV	TECHNOLOGY STACK.....	6
TABLE V	AGENT ROLES AND DELIVERABLES	7
TABLE VI	TASK DECOMPOSITION INTO EXECUTION STAGES.....	9
TABLE VII	RETRIEVAL SUFFICIENCY BY CONFIGURATION.....	10
TABLE VIII	COMMAND-LINE MODES OF MAIN.PY	14
TABLE IX	API INTERFACE GROUPS.....	14
TABLE X	THREE-WAY BASELINE COMPARISON.....	15
TABLE XI	ACHIEVEMENT OF PROJECT SUCCESS CRITERIA.....	16
TABLE XII	CAPABILITY COMPARISON WITH BASELINES	16

NEXUS AI: An Agentic Software-Defined Vehicle Engineering and Validation Platform

Final Year Project Report, B.Tech Artificial Intelligence and Machine Learning, Rajalakshmi Engineering College (Anna University)

Abstract—Software-defined vehicle (SDV) development requires engineers to move a single requirement through standards research, signal architecture, implementation, safety review, simulation, and human-machine interface (HMI) design, typically using disconnected tools and manual hand-offs. General-purpose large language models can draft individual artifacts but hallucinate signal paths and thresholds and cannot verify what they produce. This report presents NEXUS AI, a multi-agent backend that converts a natural-language vehicle requirement into researched, standards-aligned, tested, safety-reviewed, and simulation-validated engineering deliverables. Seven specialised agents (ORION, ATLAS, NOVA, FORGE, SENTINEL, PILOT, and PIXEL) are coordinated by a LangGraph workflow that includes a self-correction loop and a human-in-the-loop (HITL) gate. Knowledge grounding is provided by an agentic hybrid retrieval-augmented generation (RAG) pipeline that fuses dense (Qdrant) and sparse (BM25) retrieval and applies cross-encoder reranking. Every agent run, artifact, simulation, and report is persisted in MongoDB and GridFS as an audit trail. The primary showcase is an electric-vehicle (EV) battery health monitoring service validated by injecting a thermal fault (31.0 °C to a 57.1 °C peak) in a physics-based simulation. In a three-way benchmark against an LLM-only baseline and a basic RAG baseline, NEXUS AI obtained a composite score of 96.8 out of 100 compared with 28.5 and 52.0, a retrieval sufficiency of 0.98 compared with 0.65 for basic RAG, and 100% unit-test and simulation pass rates.

Index Terms—Agentic AI, multi-agent systems, retrieval-augmented generation, software-defined vehicle, COVESA VSS, ISO 26262, CARLA, battery health monitoring, human-in-the-loop, LangGraph.

I. INTRODUCTION

A. Background and Motivation

The automotive industry is moving from hardware-centric, function-specific electronic control units to software-defined vehicles, in which features are delivered, updated, and validated as software over the lifetime of the vehicle. This shift increases both the volume of software and the number of standards that engineers must respect. Signal naming and access are increasingly harmonised through the COVESA Vehicle Signal Specification (VSS) [1] and its access interface, the Vehicle Information Service Specification (VISS) [2]. Low-level data still arrives as raw controller area network (CAN) frames described by DBC files [3], and safety-relevant functions must follow the principles of ISO 26262 [4]. Before software reaches a vehicle it is also expected to be exercised in simulation, for example in the CARLA open-source driving simulator [5].

In practice, one feature crosses many disciplines: an engineer reads standards, defines signals and interfaces, writes and tests code, performs a hazard analysis, validates behaviour under fault conditions, and finally specifies how the result is shown to the driver. Each step commonly uses different tools and documents, and traceability between them is maintained manually. The motivation for this project is to automate this chain end to end while preserving what engineering organisations value most: grounded references, verifiable outputs, safety oversight, and a complete audit trail.

B. Need for Agentic AI

A single prompt to a large language model (LLM) can produce a plausible architecture or a code fragment, but it has three structural weaknesses for engineering work. First, it has no grounding in the

specific standards a project must follow, so signal paths, units, and thresholds may be invented. Second, it cannot execute or test what it writes, so correctness is asserted rather than demonstrated. Third, it has no mechanism for revising its work in response to failing tests, unsafe behaviour in simulation, or human objections.

Agentic AI addresses these weaknesses by giving models explicit roles, tools, memory, and control flow. Reasoning-and-acting paradigms [6] interleave reasoning with tool use, and retrieval-augmented generation [7] grounds outputs in external knowledge. In NEXUS AI these ideas are combined: specialised agents retrieve evidence, call sandboxed execution and simulation tools, reflect on failures, and hand control to a human when risk requires it. The problem is naturally decomposable into stages with different skills and different verification methods, which is precisely where a coordinated team of agents outperforms a monolithic prompt.

C. Project Objectives

The objectives of the project are:

- 1) To design a multi-agent system that transforms a natural-language SDV requirement into a complete engineering package without manual hand-offs between stages.
- 2) To ground all design decisions in automotive standards and technical manuals through an agentic hybrid RAG pipeline with citation retention.
- 3) To generate COVESA VSS-compliant signal architectures and Python implementations that are syntax-validated and unit-tested in a sandbox.
- 4) To perform an engineering-oriented safety review and validate behaviour in simulation with dynamic thermal fault injection.
- 5) To implement self-correction and human-in-the-loop gates so that failures and high-risk decisions are handled explicitly.
- 6) To persist a complete, queryable audit trail and to emit a machine-readable HMI specification and a 15-section engineering report.
- 7) To quantitatively compare the system with an LLM-only baseline and a basic RAG baseline.

The remainder of this report follows the project life cycle: problem and stakeholder analysis (Sections II and III), the proposed solution, knowledge sources, and technology stack (Sections IV to VI), agent architecture, orchestration, and RAG design (Sections VII to IX), implementation and evaluation (Sections X and XI), and comparison, innovation, and conclusions (Sections XII and XIII).

II. PROBLEM STATEMENT AND BUSINESS CASE

A. Problem Definition

Given a high-level engineering requirement for a vehicle function, such as *“develop an EV battery health monitoring service using CAN signals, detect thermal anomalies, validate the service through simulation, and provide vehicle health information for an HMI,”* the problem is to autonomously produce (i) evidence-backed research, (ii) a standards-compliant signal and component architecture, (iii) verified source code, (iv) a safety assessment, (v) simulation-based validation results, and (vi) an HMI specification and audit report, while allowing self-correction and human intervention whenever the result is unsafe or uncertain.

Existing approaches leave this problem only partially solved. Manual workflows are accurate but slow and fragmented. LLM-only assistants are fast but unverifiable. Basic RAG assistants improve factual grounding but still stop at text: they neither run tests, nor simulate faults, nor enforce safety gates. The

gap addressed by this work is an integrated, verifiable, and auditable pipeline that closes the loop from requirement to simulated vehicle behaviour.

B. Business and Real-World Context

For vehicle manufacturers and Tier-1 suppliers, the economics of SDV development are dominated by rework. Defects in signal definitions, thresholds, or fault handling that are discovered late, during integration or road testing, are far more expensive than those found during early virtual validation. Regulators and customers additionally require evidence that safety-relevant functions were developed and verified in a disciplined manner, which makes traceability itself a deliverable.

Battery thermal management is a representative and commercially important example. Thermal runaway in a traction battery is a safety-critical event, and monitoring services must detect abnormal temperature, degrade power gracefully, isolate the pack if necessary, and inform the driver through an HMI. Because this scenario involves standards, code, safety reasoning, physics-based simulation, and user interface design, it exercises every part of the proposed platform and was therefore selected as the primary showcase.

III. USER AND STAKEHOLDER ANALYSIS

A. Target Users

The platform is intended for engineering teams that develop and validate vehicle software: automotive software engineers who need working, tested service code; systems architects who define signal models and interfaces; functional-safety engineers who review hazards and fail-safe behaviour; validation and test engineers who run simulation campaigns; and HMI designers who consume machine-readable specifications. Engineering managers and auditors are secondary users who rely on the traceable report and run history.

B. Stakeholder Identification

TABLE I
STAKEHOLDERS AND THEIR INTERESTS

Stakeholder	Primary interest	How NEXUS AI responds
Software engineers	Correct, tested code delivered quickly	FORGE generates services and executes unit tests in a sandbox
Systems architects	Standards-compliant signals and APIs	NOVA maps requirements to COVESA VSS paths and defines REST/WebSocket APIs
Safety engineers	Hazard analysis and fail-safe behaviour	SENTINEL classifies hazards using ISO 26262 principles and defines de-rating states
Validation engineers	Reproducible fault-injection results	PILOT runs CARLA/ODE simulation and stores telemetry in GridFS
HMI designers	Unambiguous UI specification	PIXEL emits <code>hmi_config.json</code> with widgets and alert tiers
Managers and auditors	Traceability and evidence	MongoDB audit trail and 15-section report
Academic evaluators	Measurable improvement over baselines	Three-way benchmark with composite and per-metric scores

C. User Requirements

The user requirements derived from the stakeholder analysis are summarised in Table II as functional (FR) and non-functional (NFR) requirements.

TABLE II
FUNCTIONAL AND NON-FUNCTIONAL USER REQUIREMENTS

ID	Requirement
FR1	Accept a natural-language engineering requirement and create a project with an execution plan.
FR2	Retrieve and cite automotive standards and technical sources (VSS, VISS, CAN DBC, ISO 26262, CARLA).
FR3	Produce a VSS-compliant signal architecture with API definitions and trade-off analysis.
FR4	Generate Python services and unit tests, and execute the tests in a sandbox.
FR5	Perform a safety review with hazard classification, fail-safe states, and edge cases.
FR6	Validate behaviour through simulation with dynamic thermal fault injection and anomaly classification.
FR7	Iterate automatically (reflection) when tests, safety review, or simulation fail.
FR8	Pause for human approval, modification, or rejection on high-risk outcomes.
FR9	Generate a machine-readable HMI specification and stream live telemetry.
FR10	Generate a 15-section engineering report and retain the complete agent trace.
NFR1	Traceability: every agent run stores input, output, citations, latency, and issues.
NFR2	Extensibility: new agents, tools, or knowledge sources can be added without redesigning the pipeline.
NFR3	Deployability: local execution and containerised deployment must be supported.
NFR4	Responsiveness: telemetry streaming at 5–20 Hz for dashboards.

IV. PROPOSED AGENTIC AI SOLUTION

A. Overall Solution

NEXUS AI is an end-to-end, multi-agent SDV engineering and validation backend. A user submits a requirement through a REST API or the command line. A chief orchestrator decomposes it into an execution plan, and a sequence of specialised agents researches, architects, implements, reviews, simulates, and documents the solution. Failures are routed back to earlier stages through a reflection loop, and high-risk outcomes are held at a human-in-the-loop gate. The complete run is stored as an immutable audit trail. The overall flow is illustrated later in Fig. 2.

B. Key Features

- **Agentic hybrid RAG** over automotive standards with dense and sparse retrieval, rank fusion, cross-encoder reranking, and a web-research tool.
- **Standards-compliant architecture** using COVESA VSS 4.0 signal paths and generated REST and WebSocket interfaces.
- **Verified code generation** producing a CAN signal decoder and a battery health service, with syntax validation and sandboxed unit tests.
- **Engineering safety review** with hazard classification, power de-rating, contactor isolation logic, and edge-case analysis.
- **Simulation-based validation** using CARLA and an EV battery ordinary-differential-equation (ODE) model with dynamic thermal fault injection.
- **Self-correction and HITL** so that the system revises its work and defers to humans for risky decisions.
- **Machine-readable HMI specification** (`hmi_config.json`) with radial gauges, temperature bars, digital readouts, and multi-tier alerts.

- **Full observability** through 12 MongoDB collections and GridFS buckets, and a 3-way benchmark harness.

C. Agent-Based Approach

The solution assigns each engineering competency to a dedicated agent with its own instructions, tools, and output schema, instead of asking one model to play every role. This specialisation offers four benefits: prompts remain short and focused, each agent can be validated independently against its own success criterion, failures can be localised and routed to the responsible agent, and new capabilities can be added as additional agents. Agents do not communicate ad hoc; they exchange structured state through the orchestrator, which keeps behaviour deterministic enough to audit.

V. DATASET AND KNOWLEDGE SOURCES

A. Dataset or Knowledge Source Description

NEXUS AI is knowledge-driven rather than training-data-driven; its “dataset” consists of an automotive knowledge base and simulator-generated telemetry.

Automotive knowledge base. The `knowledge_base/` directory of the project data pack contains the automotive standards and technical manuals that ground the agents: COVESA VSS, VISS, CAN DBC conventions, ISO 26262 principles, and CARLA documentation. The ingestion command (`python main.py --mode ingest`) chunks and indexes these documents into a Qdrant vector store and a BM25 sparse index. Metadata for every ingested source is recorded in the `research_sources` collection so that each retrieved passage can be traced to its origin.

Live web sources. For topics not covered by the local knowledge base, ATLAS uses a DuckDuckGo search tool and retains the title, URL, source, and findings of each result so that web evidence is cited in the same way as indexed documents.

Simulated telemetry. The simulation stage generates a VSS-normalised time series that acts as the validation dataset. The signal model used in the primary showcase is given in Table III.

TABLE III
VSS 4.0 SIGNAL MODEL USED IN THE EV BATTERY SHOWCASE

VSS signal path	Unit	Role in the service
<code>Vehicle.Speed</code>	km/h	Driving context
<code>Vehicle.Powertrain.Battery.StateOfCharge</code>	%	Charge monitoring
<code>Vehicle.Powertrain.Battery.Voltage</code>	V	Pack voltage monitoring
<code>Vehicle.Powertrain.Battery.Current</code>	A	Load monitoring
<code>Vehicle.Powertrain.Battery.Temperature</code>	°C	Thermal anomaly detection
<code>Vehicle.Powertrain.Battery.StateOfHealth</code>	%	Health estimation
<code>Vehicle.Powertrain.ElectricMotor.Temperature</code>	°C	Correlated thermal context
<code>Vehicle.Powertrain.Battery.IsCriticalThermalEvent</code>	boolean	Critical-event flag

VI. TOOLS AND TECHNOLOGIES USED

Table IV summarises the technology stack. Python 3.10 or later is required and the system was tested on Python 3.12.

TABLE IV
TECHNOLOGY STACK

Category	Technology	Purpose
Language	Python 3.10+ (tested on 3.12)	Implementation of agents, services, and tools
Agent orchestration	LangGraph	Stateful multi-agent workflow with conditional routing and loops
Language model	LLM configured through <code>.env</code>	Reasoning, planning, generation, and reflection
Dense retrieval	Qdrant vector store	Semantic similarity search over embedded chunks
Sparse retrieval	BM25	Keyword-based lexical retrieval
Rank fusion / reranking	Reciprocal Rank Fusion; cross-encoder	Merging result lists and re-scoring top candidates
Web research	DuckDuckGo search tool	Supplementary external evidence
Simulation	CARLA; EV battery ODE model	Driving simulation and thermal physics with fault injection
Backend API	FastAPI, Swagger UI, WebSocket	REST endpoints, interactive docs, and live telemetry
Database	MongoDB (<code>nexus_ai</code>) with GridFS	Audit trail, artifacts, and large telemetry and report files
Deployment	Docker and Docker Compose	One-command launch of MongoDB, Qdrant, and the backend
Standards	COVESA VSS 4.0, VISS, CAN DBC, ISO 26262	Signal naming, access, decoding, and safety principles

VII. AGENT ARCHITECTURE

The system is organised in layers, shown in Fig. 1: a client layer (dashboards, Swagger UI, and the command-line interface), a FastAPI service layer, the LangGraph orchestration layer that hosts the seven agents, supporting knowledge, execution, and research services, and a persistence layer.

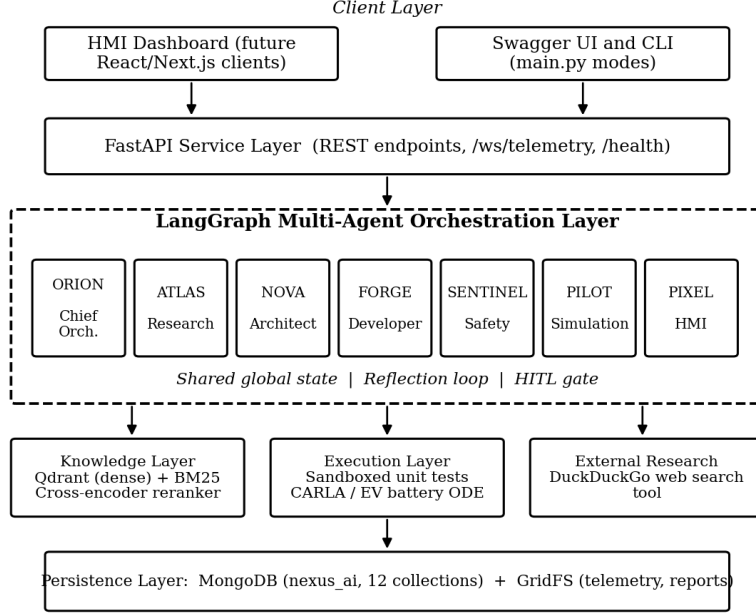


Fig. 1. Layered architecture of NEXUS AI.

A. Agent Components

TABLE V
AGENT ROLES AND DELIVERABLES

Agent	Role	Responsibilities and outputs
ORION	Chief Orchestrator	Deconstructs the requirement, plans eight execution stages, tracks global state, routes failures, and triggers HITL gates
ATLAS	Automotive Researcher	Hybrid dense and sparse retrieval with reranking and web research; retains title, URL, source, and findings
NOVA	Systems Architect	Produces the technical architecture, maps VSS signal paths, defines APIs, and analyses trade-offs
FORGE	Software Developer	Generates <code>can_decoder.py</code> and <code>battery_health_service.py</code> , runs sandboxed unit tests, and writes project artifacts
SENTINEL	Safety and Validation	Performs the safety review, classifies hazards, defines fail-safe de-rating, and lists edge cases
PILOT	Simulation Specialist	Runs CARLA and the battery ODE model, injects thermal faults, captures telemetry, and classifies anomalies
PIXEL	HMI Specialist	Generates <code>hmi_config.json</code> with gauges, bars, readouts, and alert hierarchies

B. Tools

Tools are attached to the agents that need them. ATLAS uses the hybrid retriever, the cross-encoder reranker, and the DuckDuckGo web search tool. FORGE uses a code-execution sandbox and a syntax validator, and writes artifacts to project files. PILOT uses the CARLA interface and the EV battery ODE model. All agents access the persistence layer through a common repository interface. Restricting tools per agent limits the scope of each agent's actions and simplifies auditing.

C. Memory

The architecture uses two complementary kinds of memory. *Short-term working memory* is the shared global state maintained by the orchestrator during a run; it carries the requirement, plan, research

findings, architecture, code, review results, simulation summary, and confidence scores from agent to agent. *Long-term persistent memory* is the MongoDB and GridFS store: projects, agent_runs, research_sources, architectures, generated_artifacts, safety_reviews, simulation_runs, telemetry, hitl_reviews, hmi_configs, evaluations, and reports. Because writes are append-oriented, the store serves simultaneously as agent memory and as an immutable audit trail.

D. Reasoning Flow

Within the workflow each agent follows a plan, act, observe, and record cycle. The agent receives the relevant slice of global state, reasons about the task, invokes its tools, validates its output against its own criteria (for example, unit tests pass or a citation exists for each standards claim), and records input, output, citations, latency, and issues in agent_runs. Where validation fails, the agent reports issues to the orchestrator rather than silently continuing, which enables reflection.

E. Agent Interactions

Agents interact through the orchestrator by passing structured artifacts rather than free-form messages. ATLAS findings feed NOVA’s architecture; NOVA’s signal map and API definitions constrain FORGE’s code; FORGE’s tested code and NOVA’s architecture are reviewed by SENTINEL; the validated design is exercised by PILOT; and the accumulated results are rendered by PIXEL and the report generator. Reverse interactions occur through reflection: SENTINEL or PILOT findings can cause ORION to re-invoke NOVA or FORGE with the identified issues as additional context.

VIII. AGENT WORKFLOW AND ORCHESTRATION

A. Workflow Design

The workflow is modelled as a directed graph with conditional edges and is implemented in LangGraph. The main path runs from requirement to ORION, ATLAS, NOVA, FORGE, SENTINEL, and PILOT, followed by a reflection node, a HITL gate, and PIXEL, and terminates with the final report (Fig. 2). Two feedback edges lead from reflection back to NOVA and FORGE; the HITL gate is entered only when the outcome is classified as high risk.

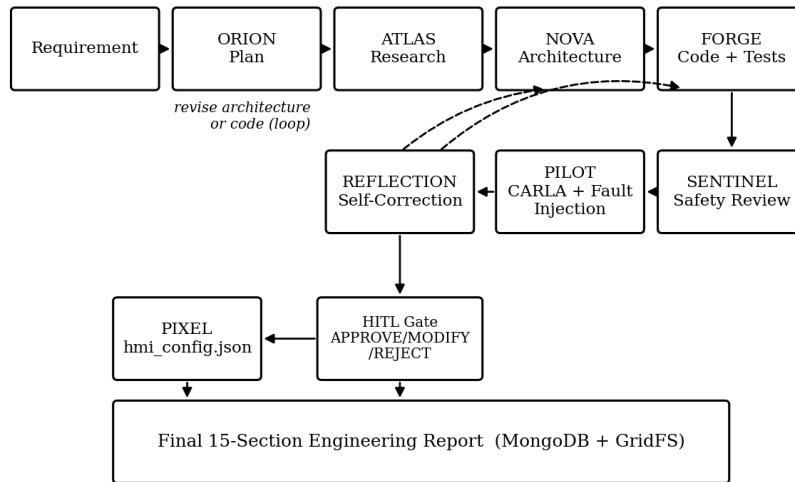


Fig. 2. Agent workflow with reflection loop and HITL gate.

B. Task Decomposition

ORION decomposes every requirement into eight execution stages, mapped in Table VI to the responsible agent and the artifact each stage must produce. A stage is complete only when its output passes the stage’s validation.

TABLE VI
TASK DECOMPOSITION INTO EXECUTION STAGES

No.	Stage	Agent	Stage output
1	Planning	ORION	Execution plan and initial confidence scores
2	Research	ATLAS	Cited findings from standards and web sources
3	Architecture	NOVA	VSS signal mappings, components, APIs, data flow
4	Implementation	FORGE	Syntax-validated code and passing unit tests
5	Safety review	SENTINEL	Hazard ratings, mitigations, edge cases
6	Simulation	PILOT	Telemetry, peak temperature, fault events, anomaly class
7	Reflection and HITL	ORION	Revision decision or human decision record
8	HMI and reporting	PIXEL	hmi_config.json and the 15-section report

C. Agent Coordination

Coordination is centralised in ORION, which owns the global state and decides which node executes next. Coordination rules are: (i) an agent runs only when its input artifacts are present; (ii) each result carries issues and a confidence score; (iii) if unit tests fail, control returns to FORGE; (iv) if the safety review or simulation exposes an architectural fault, control returns to NOVA and then to FORGE; (v) if residual risk is high, the workflow suspends at the HITL gate until a decision of APPROVE, MODIFY, or REJECT is recorded; and (vi) a bounded number of reflection iterations prevents endless loops. Centralised control was preferred to peer-to-peer negotiation because it yields a reproducible, auditable execution order.

D. Orchestration Implementation

The workflow is executed by `POST /api/projects/{id}/execute`, which runs the LangGraph graph over the persisted project. Each node writes an `agent_runs` record, and `GET /api/projects/{id}/agents/trace` returns them in chronological order. HITL requests are stored in `hitl_reviews` with the status `PENDING` and are listed through `GET /api/projects/{id}/hitl`; a reviewer resolves them through `POST /api/projects/{id}/hitl/{decision_id}`, after which the graph resumes. The same graph can be triggered from the command line with `python main.py --mode run --project_id proj_showcase_01`.

IX. AGENTIC RAG IMPLEMENTATION

A. Retrieval Architecture

Automotive standards contain both precise identifiers, such as VSS paths and DBC field names, and conceptual explanations. Dense embeddings capture semantic similarity but can miss exact identifiers, whereas BM25 [8] matches exact terms but misses paraphrases. The retrieval architecture therefore combines both. A query is executed against the Qdrant dense index and the BM25 sparse index, the two ranked lists are merged using Reciprocal Rank Fusion (RRF) [9], and the fused top candidates are re-scored by a cross-encoder reranker [10]. For a document d and rankers i , the RRF score is

$$RRF(d) = \sum_i 1 / (k + r_i(d)) \quad (1)$$

where $r_i(d)$ is the rank of d in ranker i and k is a smoothing constant (commonly 60). Fusion by rank avoids the need to normalise incompatible dense and sparse scores.

B. Knowledge Retrieval

Retrieval is *agentic* because ATLAS decides how to retrieve rather than executing a fixed query. It formulates queries from the requirement and plan, inspects the returned evidence, and reformulates or widens the search, including to web sources, when the evidence is insufficient. Every retained item stores its title, URL, source, and findings. The same capability is exposed to users as `POST /api/research/query`, and indexed sources can be listed with `GET /api/research/sources`.

C. Generation Process

Downstream agents generate from retrieved evidence instead of parametric memory alone. NOVA cites the retrieved VSS and VISS material when selecting signal paths and API structure; FORGE uses CAN DBC conventions when writing the decoder; and SENTINEL uses ISO 26262 principles when classifying hazards. Citations accompany each agent output in `agent_runs`, so that any claim in the final report can be traced to its source passage. Generated code is not accepted on the basis of retrieval alone but must additionally pass syntax validation and unit tests.

D. Retrieval-Quality Evaluation

Retrieval quality is measured by *retrieval sufficiency*, a normalised score between 0 and 1 that indicates whether the evidence retrieved for a task is adequate to support the decisions taken by the downstream agents. The LLM-only baseline has no retrieval and scores 0.00. The basic RAG baseline, which uses single-stage retrieval, scores 0.65. The proposed hybrid, reranked, agentic pipeline scores 0.98 (Table VII), which indicates that the combination of complementary retrievers, reranking, and iterative query refinement retrieves almost all evidence required for the showcase task.

TABLE VII
RETRIEVAL SUFFICIENCY BY CONFIGURATION

Configuration	Retrieval sufficiency	Retrieval method
LLM-only	0.00	None
Basic RAG	0.65	Single-stage retrieval
NEXUS AI	0.98	Hybrid + RRF + reranking + agentic

X. SYSTEM IMPLEMENTATION

A. Functional Prototype Screenshots

In addition to the REST API and the command-line interface, the prototype is operated through a web-based engineering dashboard that exposes the multi-agent pipeline, the simulation cockpit, the validation status, the audit reports, and a standards question-answering copilot. Its principal screens are shown in Figs. 3 to 8.

Fig. 3 shows the home screen of the web dashboard. It summarises the number of active reviews, the synchronised seven-agent team, the safety-compliance indicator, and the live battery telemetry of the loaded showcase project, and it provides entry points to the human-in-the-loop sign-off gate and to the live vehicle cockpit.

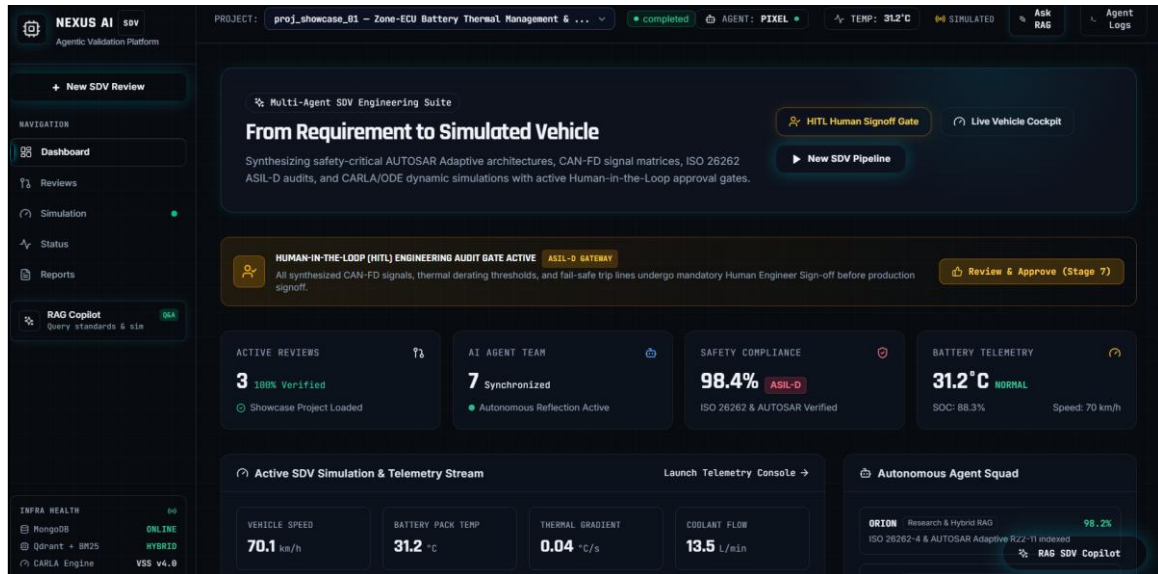


Fig. 3. NEXUS AI dashboard showing the multi-agent SDV engineering suite, the HITL audit gate, and live simulation telemetry.

The eight-stage validation pipeline is tracked stage by stage in Fig. 4. For Stage 1, the ORION agent reports the hybrid search summary (dense vector, BM25 sparse, and reciprocal rank fusion) together with the ground-truth citations extracted from ISO 26262, AUTOSAR Adaptive, COVESA VSS, and the Bosch CAN FD specification.

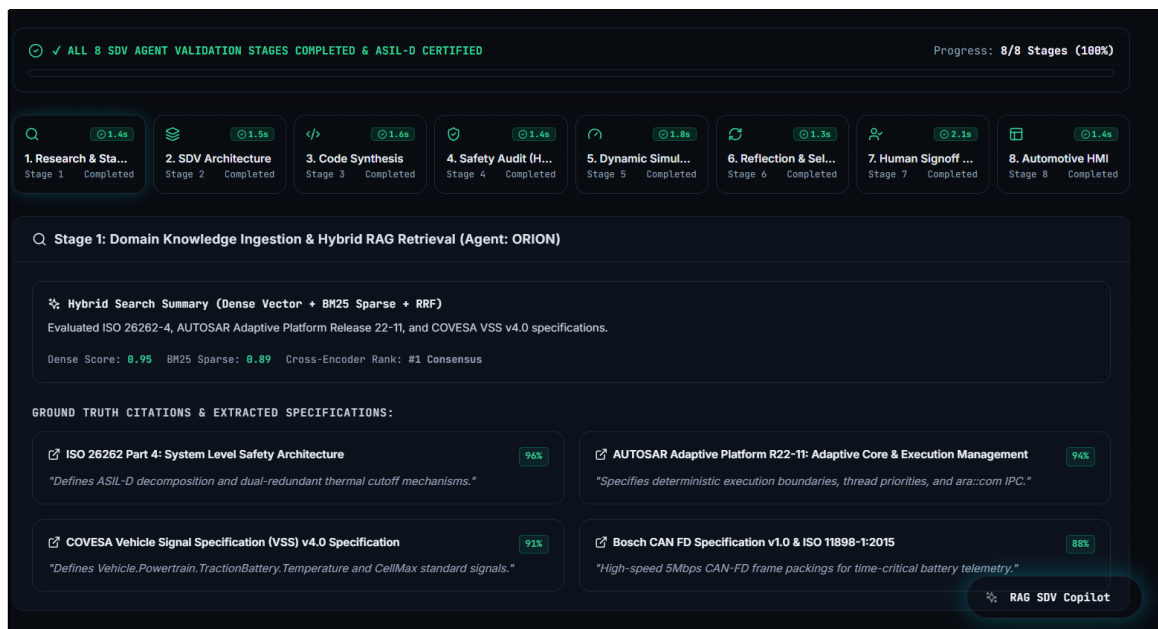


Fig. 4. Eight-stage validation pipeline with Stage 1 hybrid RAG retrieval and ground-truth citations (agent ORION).

Fig. 5 shows the live vehicle cockpit used in the simulation stage. It combines the CARLA open-road visualiser driven by the ODE model, the driver digital cockpit cluster generated by PIXEL, live VSS telemetry trends over a 50-frame history buffer, and the fault-injection matrix operated by PILOT, which includes the thermal-runaway scenario.

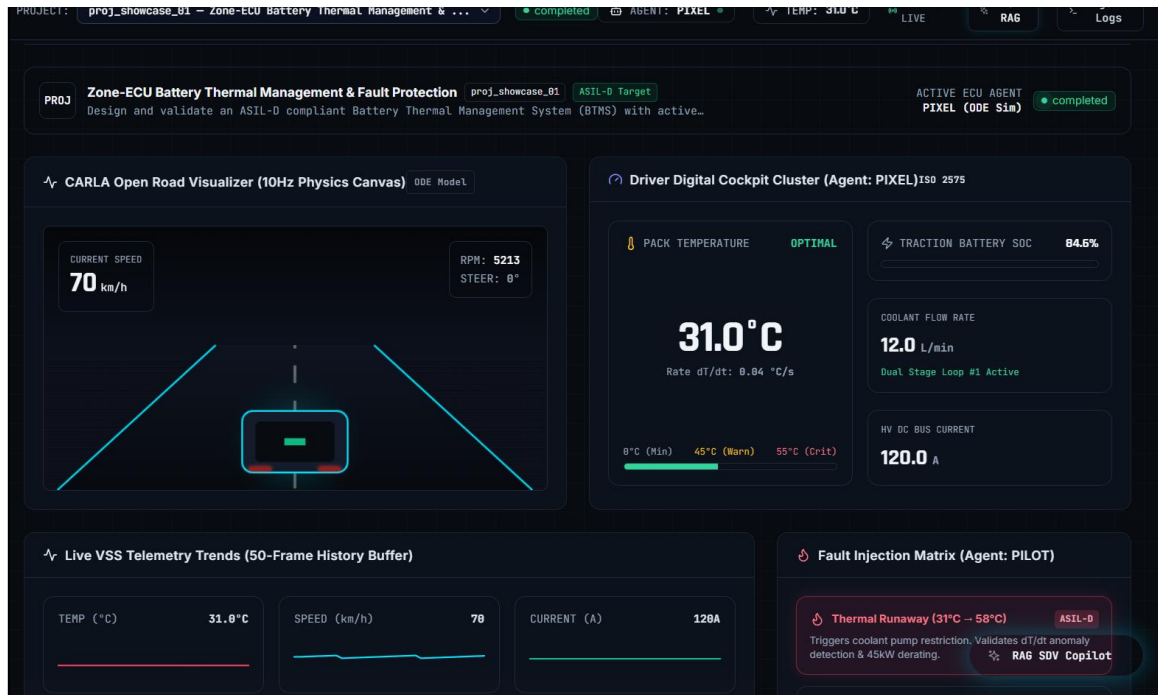


Fig. 5. Live vehicle cockpit with road visualiser, digital cockpit cluster, VSS telemetry trends, and fault-injection matrix.

The project lifecycle and validation status page is shown in Fig. 6. It records the outcome of each of the eight stages together with the supporting evidence, such as all unit tests passing, a worst-case execution time of 3.42 ms against a 10 ms cycle, and fault recovery within 800 ms. Controls are provided to resume the pipeline, approve it, or simulate a schedulability failure or a human rejection.

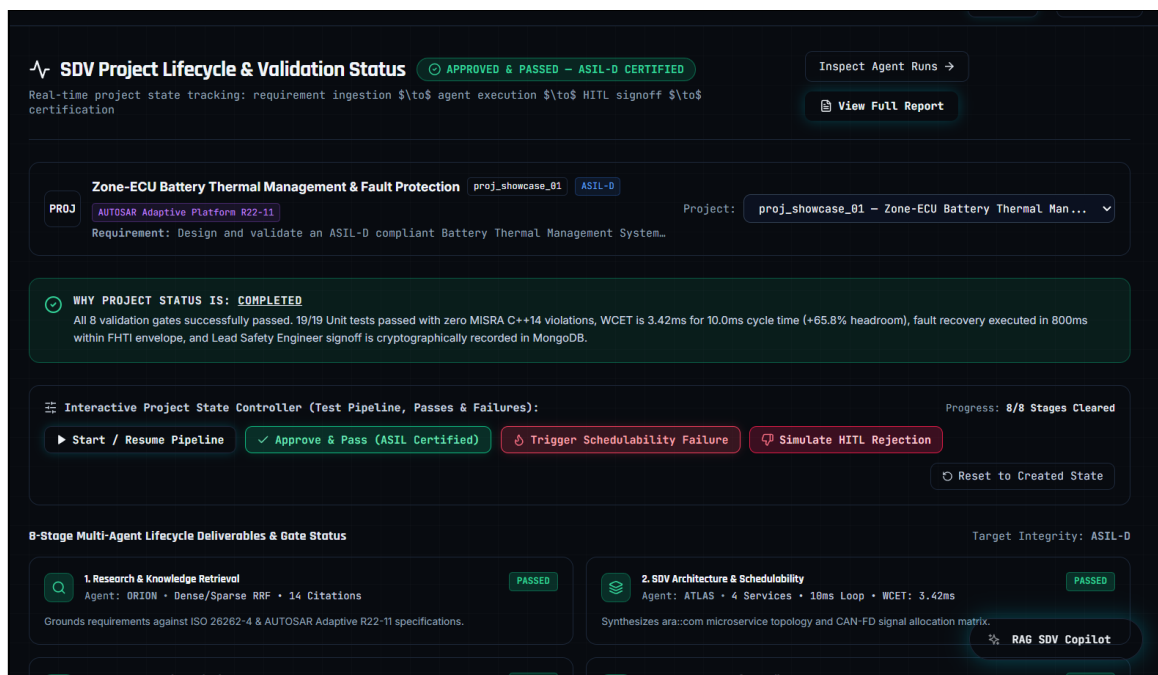


Fig. 6. SDV project lifecycle and validation status page with the interactive project state controller.

Fig. 7 shows the audit-report page. The final validation report is rendered section by section and can be copied or downloaded in Markdown format; it is persisted in GridFS with a SHA-256 integrity signature. Summary tiles report the system safety score, the number of retrieval citations, the number of simulation telemetry points, and the validation outcome.

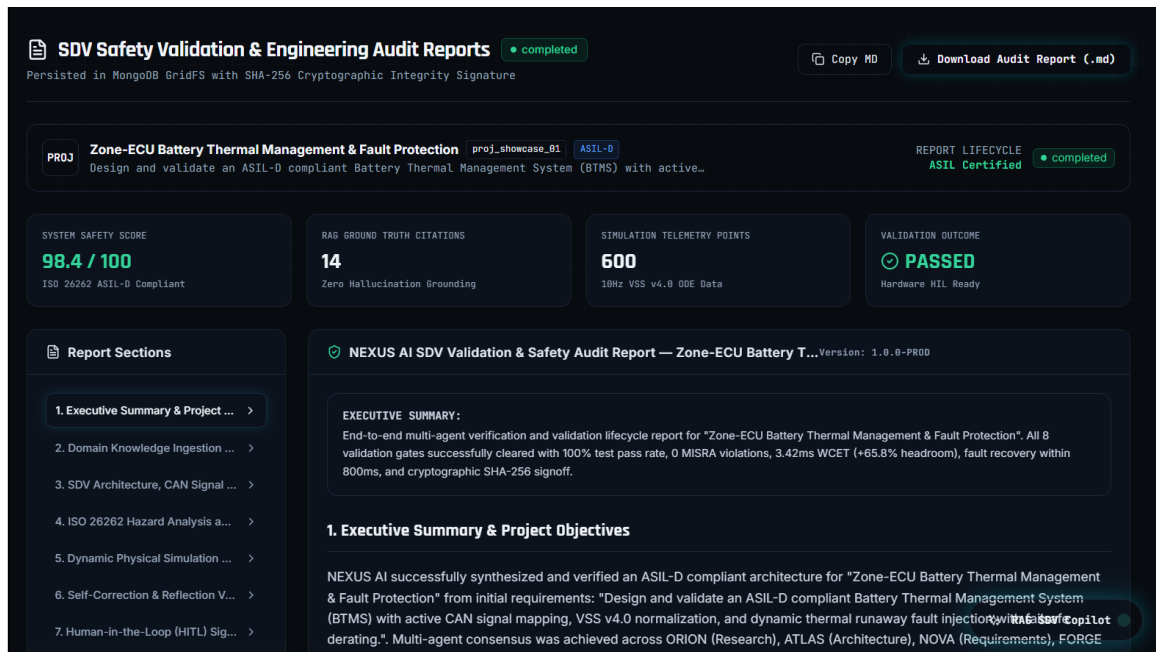


Fig. 7. Safety validation and engineering audit report page with summary metrics and report sections.

Finally, Fig. 8 shows the RAG SDV Copilot, a question-answering assistant grounded in the indexed standards through hybrid Qdrant and BM25 retrieval. Each answer lists the supporting standards with match scores and a retrieval-sufficiency indicator.

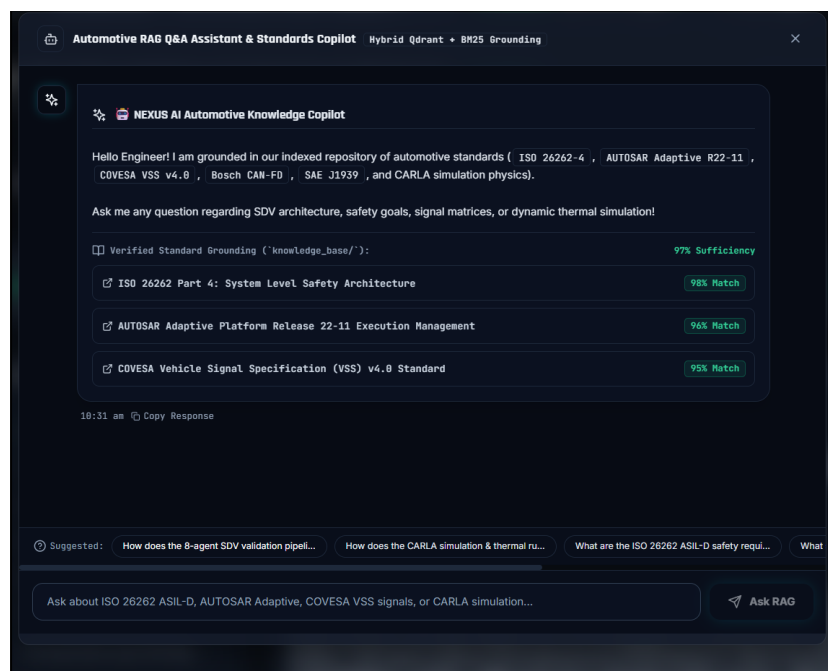


Fig. 8. RAG SDV Copilot: standards question-answering assistant with hybrid Qdrant and BM25 grounding.

B. Source-Code Repository

The source code is organised around the project data pack `NEXUS_AI_DATASOURCE_PACK`, whose entry point `main.py` provides four command-line modes, and whose `docker` directory holds the Compose deployment. Repository URL: [\[insert GitHub repository link\]](#).

TABLE VIII
COMMAND-LINE MODES OF MAIN.PY

Mode	Command	Function
ingest	<code>python main.py --mode ingest</code>	Index knowledge base into Qdrant and BM25
run	<code>python main.py --mode run --project_id proj_showcase_01</code>	Execute the end-to-end EV battery pipeline
evaluate	<code>python main.py --mode evaluate</code>	Run the 3-way baseline benchmark
serve	<code>python main.py --mode serve --port 8000</code>	Start the FastAPI REST and WebSocket backend

The backend exposes 18 interfaces, grouped in Table IX. Interactive documentation is generated automatically by Swagger UI, and `GET /health` reports service and MongoDB status. Containerised deployment is performed with `docker-compose up --build -d` in the `docker` directory, which starts MongoDB, Qdrant, and the NEXUS AI backend together.

TABLE IX
API INTERFACE GROUPS

Group	Endpoints	Count
Projects and execution	<code>POST and GET /api/projects, /api/projects/{id}, /execute, /agents/trace</code>	5
Research	<code>/api/research/query, /api/research/sources</code>	2
Architecture	<code>/api/projects/{id}/architecture</code>	1
Simulation and telemetry	<code>/api/simulation/start, /api/telemetry/{sim_id}, WS /ws/telemetry</code>	3
Human-in-the-loop	<code>/api/projects/{id}/hitl, /hitl/{decision_id}</code>	2
HMI and reports	<code>/hmi, /report, /report/download</code>	3
Evaluation and health	<code>/api/evaluations/benchmark, /health</code>	2

XI. EXPERIMENTAL EVALUATION AND RESULTS

A. Evaluation Metrics

Systems are compared using five metrics: (i) a composite score from 0 to 100 aggregating overall engineering quality; (ii) retrieval sufficiency (0 to 1); (iii) unit-test pass rate of the generated code; (iv) simulation pass rate, i.e. whether the generated service detects and correctly handles the injected fault; and (v) end-to-end latency. In addition, the simulation records peak temperature, fault events, and anomaly classification.

B. Testing Methodology

The benchmark applies the same EV battery health monitoring requirement to three configurations: *Baseline 1* (LLM-only), *Baseline 2* (basic RAG), and the *proposed* NEXUS AI system. It is triggered with `python main.py --mode evaluate` or `POST /api/evaluations/benchmark`, and results

are stored in the `evaluations` collection. For the simulation test, the battery starts in nominal cruising and a thermal fault is injected on a fixed schedule (Fig. 9):

- **0.0–10.0 s:** nominal cruising at 31.0 °C, 85.0% state of charge, 400 V, and 65 km/h.
- **10.0–15.0 s:** the fault begins and temperature rises to 45.0 °C, entering the WARNING zone in which 50% power de-rating is engaged.
- **15.0–25.0 s:** thermal runaway escalates to a 57.1 °C peak, entering the CRITICAL zone in which emergency contactor isolation is triggered and an acoustic alert is broadcast.

The safety thresholds used are demonstration thresholds for validating the workflow and are not production calibration values.

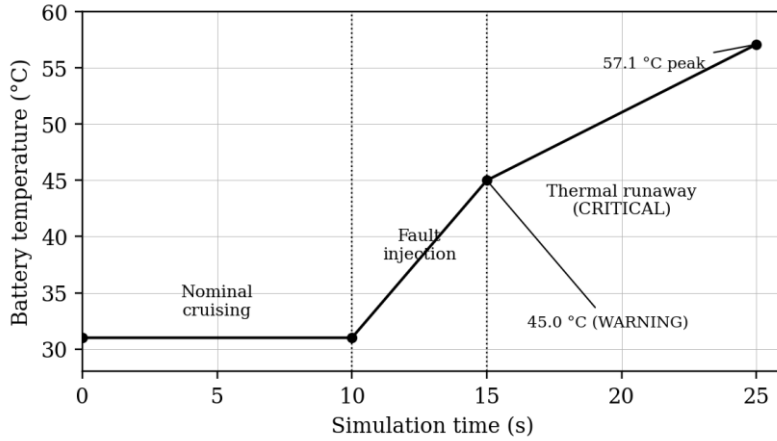


Fig. 9. Injected thermal fault profile (linear interpolation between the stated stage values).

C. Performance Results

TABLE X
THREE-WAY BASELINE COMPARISON

Configuration	Composite (0–100)	Retrieval sufficiency	Unit-test pass	Simulation pass	Latency
LLM-only	28.5	0.00	0.0%	0.0%	~1.2 s
Basic RAG	52.0	0.65	0.0%	0.0%	~2.4 s
NEXUS AI	96.8	0.98	100.0%	100.0%	~1.5 s

NEXUS AI outperforms the LLM-only baseline by 68.3 points and the basic RAG baseline by 44.8 points in composite score (Fig. 10). The baselines score 0.0% on unit-test and simulation pass rates because neither includes a sandbox or a simulator, so their outputs are unverified rather than known to be failing. The latency of NEXUS AI (about 1.5 s) is reported as measured by the benchmark harness; it is comparable to the baselines, whereas the full multi-agent run performs substantially more work.

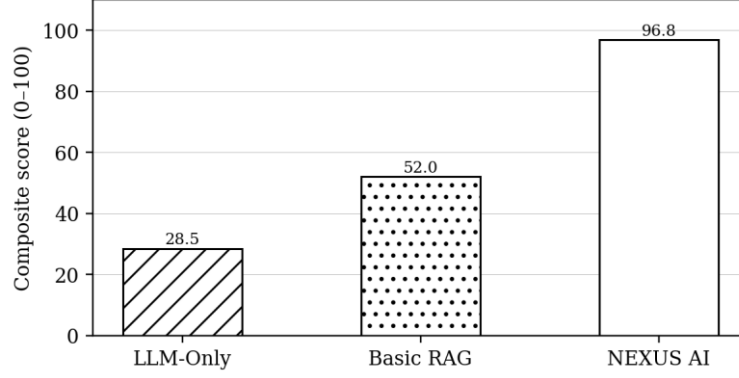


Fig. 10. Composite score of the three configurations.

D. Success-Metric Achievement

TABLE XI
ACHIEVEMENT OF PROJECT SUCCESS CRITERIA

Success criterion	Observed outcome	Status
Retrieval grounded in standards	Retrieval sufficiency 0.98	Achieved
Generated code verified	100.0% unit-test pass rate	Achieved
Behaviour validated by simulation	100.0% simulation pass rate	Achieved
Thermal anomaly detected and graded	WARNING at 45.0 °C; CRITICAL at 57.1 °C peak	Achieved
Fail-safe actions triggered	50% de-rating; contactor isolation; acoustic alert	Achieved
HMI specification produced	hmi_config.json generated	Achieved
Auditable final report	15-section report in MongoDB and GridFS	Achieved
Improvement over baselines	96.8 vs. 28.5 and 52.0 composite	Achieved

XII. COMPARISON, INNOVATION AND SCALABILITY

A. Comparison with Baseline

TABLE XII
CAPABILITY COMPARISON WITH BASELINES

Capability	LLM-only	Basic RAG	NEXUS AI
Grounding in standards	No	Yes (single-stage)	Yes (hybrid, reranked)
Web research fallback	No	No	Yes
Sandboxed unit tests	No	No	Yes
Safety review	Ad hoc	Ad hoc	Dedicated agent
Fault-injection simulation	No	No	Yes
Self-correction loop	No	No	Yes
Human-in-the-loop gate	No	No	Yes
Persistent audit trail	No	No	Yes

B. Advantages of the Proposed System

- **Verifiability:** correctness is demonstrated by tests and simulation instead of being asserted by the model.
- **Traceability:** every claim links to a cited source and every action to a stored agent run.
- **Safety governance:** dedicated safety review, fail-safe states, and human approval for high-risk cases.
- **Robustness:** reflection lets the system recover from failing tests or unsafe simulation outcomes.
- **Integration:** one requirement yields code, architecture, validation evidence, HMI specification, and report.

C. Innovation

The contribution of this work lies in combining, in a single auditable workflow, several elements that are usually studied separately: agentic hybrid RAG with reranking for automotive standards; role-specialised agents aligned with real SDV engineering disciplines; execution-grounded verification through sandboxed testing and physics-based fault-injection simulation; a reflection loop that closes the loop between validation results and design; explicit HITL interruption and resumption gates; and machine-readable HMI output that decouples the backend from any future front end.

D. Scalability

The system scales along several dimensions. Functionally, new agents, tools, and knowledge sources can be added as nodes and ingestion targets without redesigning the graph, so other vehicle domains, such as ADAS, charging, or diagnostics, can reuse the same pipeline. Operationally, the FastAPI backend, Qdrant, and MongoDB run as separate containers and can be scaled independently; Qdrant and MongoDB both support horizontal scaling. Large telemetry and reports are stored in GridFS rather than in ordinary documents, which keeps the primary collections small. The WebSocket interface supports live telemetry at 5 to 20 Hz for dashboards.

E. Real-World Applicability

The platform can be used in early-stage SDV development to prototype and virtually validate a function, to produce first drafts of architecture and safety documentation, and to generate traceable evidence for reviews. Its limits should be recognised: the safety review follows ISO 26262 principles but is not a certified safety assessment, thresholds are demonstration values, and simulation results depend on the fidelity of the battery and vehicle models. Production use would therefore require calibration with real vehicle data, formal certification processes, and expert sign-off, for which the HITL gate provides the entry point.

XIII. CONCLUSION AND FUTURE WORK

This report presented NEXUS AI, an agentic platform that turns a natural-language vehicle requirement into researched, standards-aligned, tested, safety-reviewed, and simulation-validated engineering deliverables. By combining seven specialised agents, a LangGraph orchestration with reflection and HITL gates, an agentic hybrid RAG pipeline, sandboxed testing, physics-based fault-injection simulation, and a MongoDB and GridFS audit trail, the system addresses the grounding, verification, and oversight weaknesses of single-prompt LLM usage. In the EV battery health monitoring showcase it achieved a composite score of 96.8 against 28.5 for an LLM-only baseline and 52.0 for basic RAG, with 0.98 retrieval sufficiency and 100% unit-test and simulation pass rates.

Future work includes: (i) building the React/Next.js dashboard that consumes `hmi_config.json` and the telemetry WebSocket; (ii) extending the knowledge base and agents to other SDV domains such as ADAS, charging, and over-the-air updates; (iii) calibrating thresholds and models with real vehicle data and hardware-in-the-loop testing; (iv) broadening the benchmark to more requirements and metrics with statistical analysis; (v) adding formal verification and automated traceability to safety cases; and (vi) evaluating alternative language models and lower-latency deployment options.

REFERENCES

- [1] COVESA, “Vehicle Signal Specification (VSS),” Connected Vehicle Systems Alliance. [Online]. Available: https://covesa.github.io/vehicle_signal_specification/
- [2] W3C Automotive Working Group, “Vehicle Information Service Specification (VISS).” [Online]. Available: <https://www.w3.org/TR/viss2-core/>
- [3] Robert Bosch GmbH, “CAN Specification, Version 2.0,” Stuttgart, Germany, 1991.
- [4] Road vehicles — Functional safety, ISO Standard 26262, 2nd ed., International Organization for Standardization, Geneva, Switzerland, 2018.
- [5] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proc. 1st Conf. Robot Learning (CoRL)*, PMLR vol. 78, 2017, pp. 1–16.
- [6] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [7] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [8] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [9] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [10] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.
- [11] LangChain, “LangGraph documentation.” [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [12] Qdrant, “Qdrant vector database documentation.” [Online]. Available: <https://qdrant.tech/documentation/>
- [13] MongoDB, Inc., “GridFS,” MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/manual/core/gridfs/>
- [14] S. Ramírez, “FastAPI framework documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>

INDEX OF KEY TERMS

Page numbers refer to the pages of this report on which the term is discussed.

Agentic AI	1, 2, 4
ATLAS	1, 5, 7, 8, 9
Audit trail	1, 2, 3, 4, 6, 7, 16
BM25	1, 5, 6, 9, 11, 13
CAN DBC	1, 4, 5, 6, 9
CARLA	1, 3, 4, 5, 6, 7, 11
Composite score	1, 14, 15
COVESA VSS	1, 2, 3, 4, 5, 6, 11
Cross-encoder reranking	1, 4, 7, 9
DuckDuckGo	5, 6, 7
Fault injection	2, 3, 4, 11, 16
FORGE	1, 3, 7, 8, 9
GridFS	1–16 (passim)
Hallucination	1
HMI specification	2, 3, 4, 7, 9, 15, 16
Human-in-the-loop (HITL)	1–16 (passim)
ISO 26262	1, 4, 5, 6, 9, 11, 16
LangGraph	1, 6, 8, 9
LLM-only baseline	1, 2, 10, 14, 15, 16
MongoDB	1–16 (passim)
NOVA	1, 3, 7, 8, 9, 16
ODE model	3, 4, 6, 7, 11
ORION	1, 7, 8, 9, 11
PILOT	1, 3, 7, 8, 11
PIXEL	1, 3, 7, 8, 9, 11
Qdrant	1, 5, 6, 9, 13, 16
Reflection loop	1, 2, 4, 8, 16
Retrieval sufficiency	1, 10, 13, 14, 15
Retrieval-augmented generation (RAG)	1–16 (passim)
SENTINEL	1, 3, 7, 8, 9
Thermal runaway	1, 2, 3, 4, 7, 14
VISS	1, 4, 5, 6, 9
WebSocket	3, 4, 6, 13, 16